Warsztat 3 · Sztuczna inteligencja w grach i rozszerzonej rzeczywistości

YOLO jako czujnik w minigrze

Czyli kiedy gra powinna uwierzyć sieci neuronowej?

Autorzy: [uzupełnijcie skład zespołu]

Problem

Minigra: pokaż butelkę w oznaczonej strefie obrazu, a odblokujesz nagrodę.

- Oczy gry to gotowa sieć YOLO. Nie trenujemy jej.
- O nagrodzie decyduje **reguła napisana przez Was**.
- Zamiast kamery używamy 12 przygotowanych zdjęć, takich samych dla wszystkich zespołów.

Zdjęcie
12 gotowych obrazów

→

YOLO
klasa, ramka, ufność

→

Reguła gry
Wasz kod

→

Decyzja
nagroda lub brak nagrody

Cel: połączyć wynik sieci z regułą gry i sprawdzić, jak **próg ufności** wpływa na błędne decyzje.

Co zwraca model?



Model: yolo11n, 80 klas zbioru COCO

Pojęcie	Co oznacza
klasa	co to jest: `bottle`
ramka	gdzie to jest, w pikselach: `x1, y1, x2, y2`
ufność	jak pewny jest model, od 0 do 1: `0.83`

> Ufność to nie prawdopodobieństwo poprawności, tylko sygnał, który porównujemy z wybranym progiem.

Reguła gry

Strefa: środkowa 1/3 szerokości obrazu, na całej wysokości.

Nagroda, gdy co najmniej jedna butelka spełnia oba warunki:

1. jej ufność przekracza próg,
2. **środek** jej ramki leży w strefie.

> Jedno zdjęcie daje najwyżej jedną nagrodę.



Zielona ramka przekracza próg, pomarańczowa nie.

Próg ufności: 0,25 czy 0,60?

Decyzję gry porównujemy z etykietą referencyjną każdego zdjęcia:

	gra dała nagrodę	gra odmówiła
etykieta: nagroda	**poprawna nagroda**	**pominięta nagroda**
etykieta: brak nagrody	**błędna nagroda**	**poprawna odmowa**

Do sprawdzenia: skąd bierze się każda pomyłka? Brak detekcji, niepoprawna lokalizacja lub klasa, czy sama reguła gry?

Do przemyślenia: co bardziej szkodzi grze, nagroda bez powodu czy brak nagrody za poprawną akcję?

Narzędzia

Plik	Co zawiera
`warsztat3.ipynb`	notatnik z TODO
`data/`	12 zdjęć i etykiety
`yolo11n.pt`	gotowy model
`yolo_utils.py`	funkcje pomocnicze
`sprawdz_srodowisko.py`	test instalacji
`kamera.py`	rozszerzenie
`README.md`, `requirements.txt`	uruchomienie i wersje
`sprawozdanie_szablon.md`	pusty szablon

PYTHON 3.11+ | ULTRALYTICS · OPENCV · JUPYTER | WYSTARCZY CPU

Uruchomienie

1. `python -m venv .venv`
2. aktywacja środowiska (README)
3. `python -m pip install -r requirements.txt`
4. `python sprawdz_srodowisko.py`
5. `python -m notebook`

Na Linuksie najpierw PyTorch na CPU, zgodnie z README.

Plan pracy: macie 90 minut

0–10 min

Start

Role, sprawozdanie, ręczna ocena 2 zdjęć

10–25 min

Detekcja

Model, odczyt ramek, wyznaczenie strefy

25–55 min

Reguła gry

Środek ramki, warunek strefy, decyzja o nagrodzie

55–75 min

Eksperyment

12 zdjęć, 2 progi, przyczyny pomyłek

75–90 min

Oddanie

Wnioski i wysyłka na Teams

Sprawozdanie uzupełniacie w trakcie ćwiczenia, nie dopiero pod koniec.

Skończyliście wcześniej? Otwórzcie `kamera.py`: punkt po 3 kolejnych klatkach, następny po opuszczeniu strefy.

Wyniki i demo

Do końca 90 minut

1. Uzupełniony notatnik
2. Sprawozdanie: tabela obu progów, przyczyny pomyłek, wnioski
3. Dwa zrzuty: przykład poprawny i problematyczny

Demo: 5–6 minut na zespół

1. Decyzja minigry na dwóch zdjęciach
2. Porównanie progów 0,25 i 0,60
3. Jeden błąd detekcji lub przyznania nagrody

Powodzenia!